

Today's Agenda:-

1. Count Sort
2. Merge Sort

Starting 7:05

Count Sort

< Question > : Find the smallest number that can be formed by rearranging the digits of the given number in an array. Return the smallest number in the form of an array.

arr[] $\rightarrow$ [6 3 4 2 7 2 1]

$$0 \leq a[i] \leq 9$$

o/p $\rightarrow$ [1, 2, 2, 3, 4, 6, 7]

Simply sort the array:-

arr[] $\rightarrow$ [4 2 7 3 9 0]

o/p $\rightarrow$ [0, 2, 3, 4, 7, 9]

0 to 250



Idea -1

Use inbuilt sorting algo.

$N (\log N)$



Idea -2

Use the fact that every no. n away is a digit

arr[] $\rightarrow$ [9 1 2 5 4 2 1 2 5 8]

$$0 \leq a[i] \leq 9$$

farr $\rightarrow$

0	2	3	0	1	2	0	0	1	1
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

i

i

idx = 0

o/p :- { 1, 1, 2, 2, 2, 4 }

0	0	15	0	0	0	...	1
0	1	2	...	...	...	546	

1
2
3
4
5
6

int [] smallest Number (int [] arr) {

N

```
int freq[10];
for (i=0; i<N; i++) {
    freq[arr[i]]++;
}
idx=0;
int ans[N];
```

N+R

```
for (i=0; i<10; i++) {
    for (j=0; j<freq[i]; j++) {
        ans[idx] = i;
        idx++;
    }
}
```

10.

return ans;

T.C → O(N+R)

S.C → O(R)

10.

21 to 30

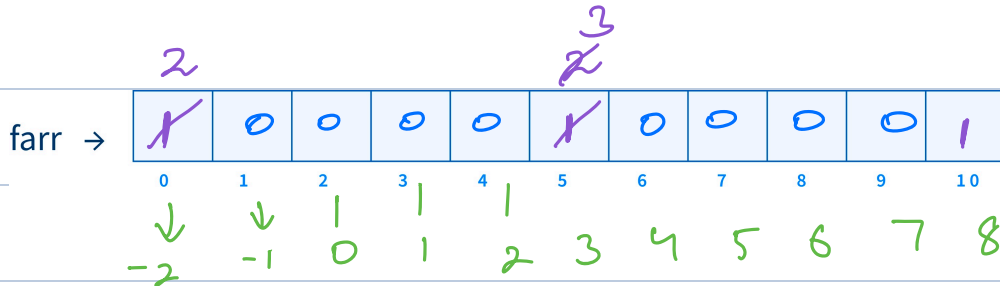
How to handle negative numbers?

i

arr[] $\rightarrow$ [-2 3 8 3 -2 3]

min:- -2; [-2, 8]

max = 8;



max - min
+ 1

$$= 8 - (-2) + 1$$
$$= 11$$

arr[i] - min

o/p:- { -2, -2, 3, 3, 3, 8 }

$$8 - (-2) = 10$$

i + min

Approach:-

1. Iterate on the array to find min & max of the array.

2. Create a farr of size (max - min + 1)
frequency array

3. Iterate on the array & generate farr ; farr [arr[i] - min] ++;

4. Iterate on the freq. array, for each value = i + min

</> Code

```
mm = findMin(arr);  
max = findMax(arr);
```

```
int arr[max - mm + 1];  
for (int i = 0; i < max - mm + 1; i++) arr[i] = 0;
```

```
int range = max - mm + 1;  
for (int i = 0; i < N; i++) {  
    arr[arr[i] - mm]++;  
}
```

```
for (int i = 0; i < range; i++)  
    for (int j = 0; j < arr[i]; j++) {  
        ans[idx] = i + mm;  
        idx++;  
    }
```

TC $\rightarrow O(N + R)$
SC $\rightarrow O(R)$

If the range of elements
to be sorted is known,
think of applying the Count
sort algorithm.

$$\underline{-2 \times 10^9 ; +2 \times 10^9}$$

Merge Two Sorted Arrays?

a[] $\rightarrow$ [2 4 7 8 12] $\rightarrow$ N 5

($-10^9 \leq \text{element} \leq 10^9$)

b[] $\rightarrow$ [3 5 6 7] $\rightarrow$ M 4.

o/p:- { 2, 3, 4, 5, 6, 7, 7, 8, 12 }.

Idea 1:-

0	1	2	3	4	5	6	7	8
2	4	7	8	12	3	5	6	7

$\downarrow$ Sorting algo.

0	1	2	3	4	5	6	7	8
2	3	4	5	6	7	7	8	12

$$N + m + (N+m) \lg(N+m)$$

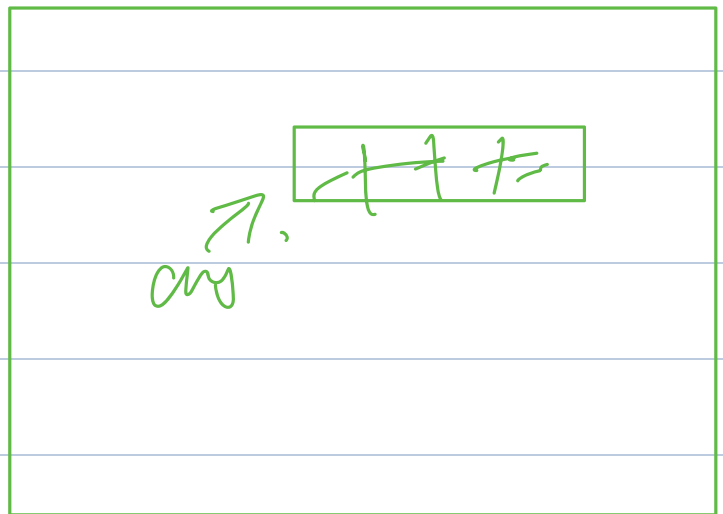
Idea 2:- Compare the 2 elements from both the arrays & keep acquiring the smallest one optimally.

a[] $\rightarrow$ [2 4 7 8 12] $\rightarrow$ N

b[] $\rightarrow$ [3 5 6 7] $\rightarrow$ M

o/p:-

0	1	2	3	4	5	6	7	8
2	3	4	5	6	7	7	8	12



</> Code

```
int[] merge2sortedArrays (int A[],  
                           int B[]) {
```

```
    N = A.size(); m = B.size();
```

```
    int ans[N+m];
```

```
    i = 0; j = 0; idx = 0;
```

```
    while (i < N & j < m) {
```

```
        if (a[i] <= b[j]) {
```

```
            ans[idx] = a[i];
```

```
            i++; idx++;
```

```
        } else {
```

```
            ans[idx] = b[j];
```

```
            j++; idx++;
```

```
        }
```

Handle j out of bound.

```
    while (i < N) {
```

```
        ans[idx] = a[i]; i++; idx++;
```

```
    }
```

Handle i out of bound.

while ($j < m$) {

} ans[idx] = b[j]; j++; idx++
}

return ans;

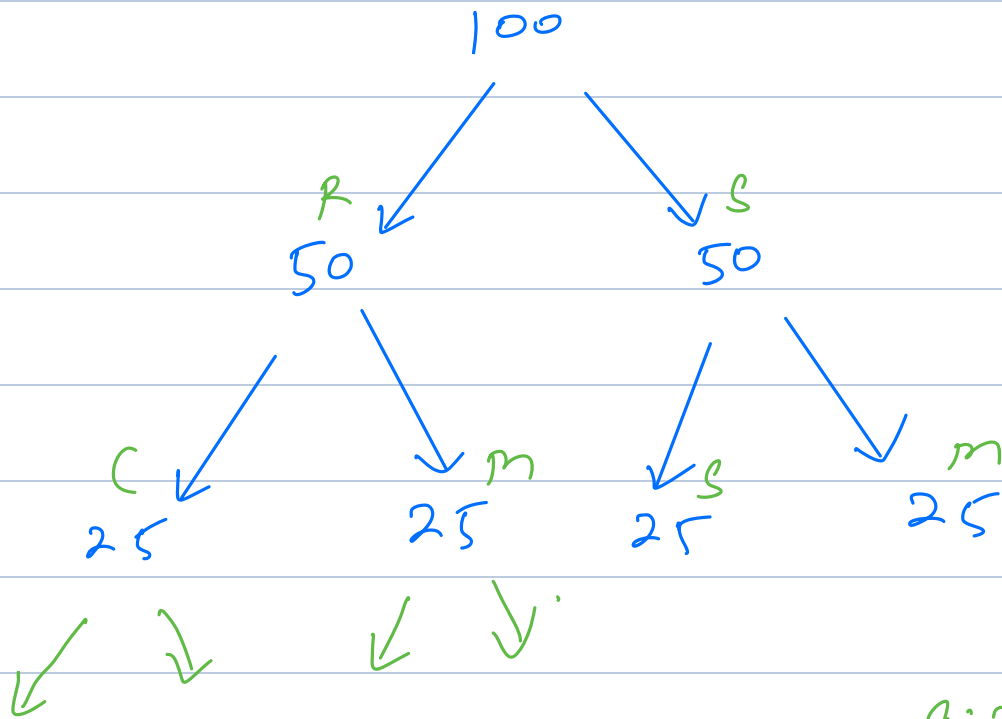
3

T.C $\rightarrow O(N+m)$

S.C $\rightarrow O(1)$

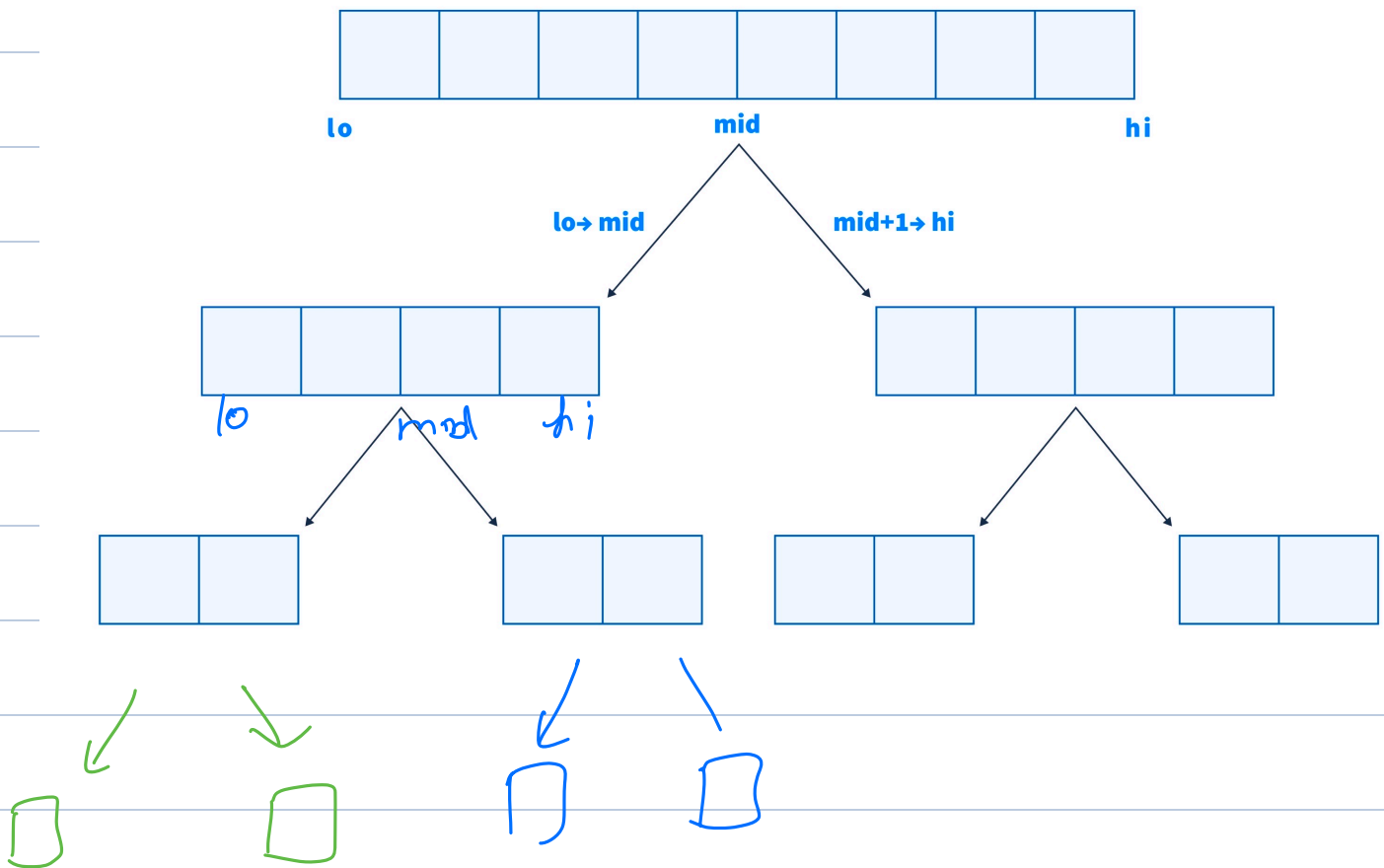
Merge Sort

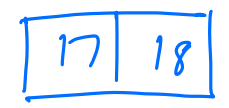
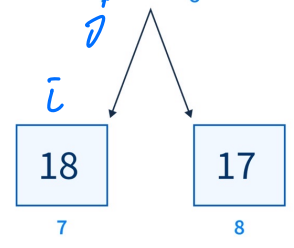
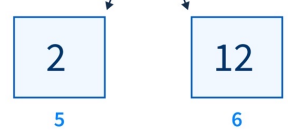
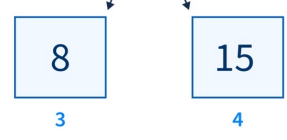
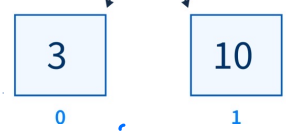
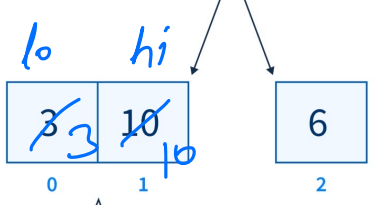
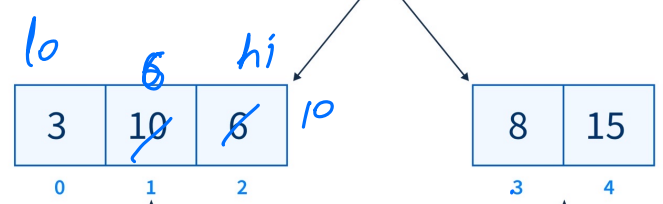
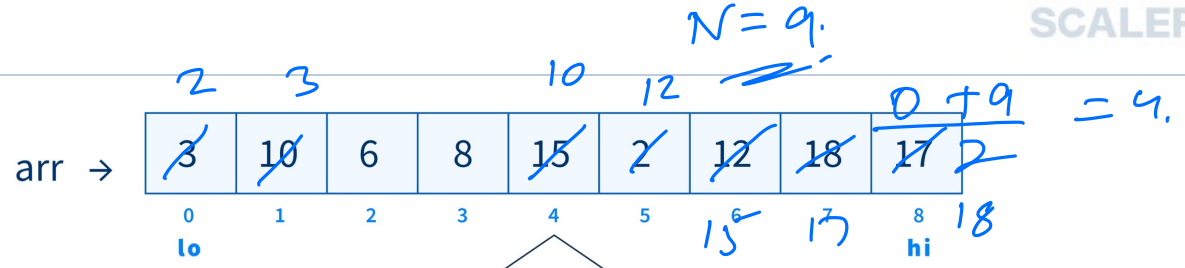
arrange An exam inviligator wants to
100 sheets roll no. wise.



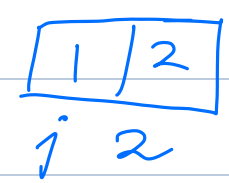
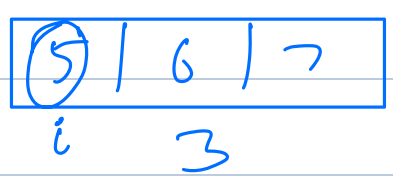
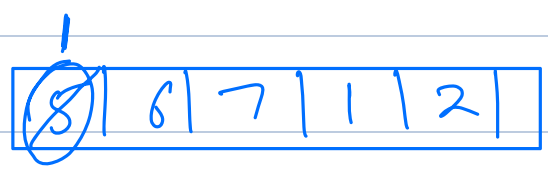
8:32:

$$\text{mid} = \frac{\text{lo} + \text{hi}}{2}$$





lo j



Code.

```
void mergeSort (int arr[],  
                int lowlo, int highhi) {  
    ↓ 0           ↓ N-1
```

```
    if (lo == hi) {  
        return;  
    }
```

```
    mid = (lo + hi) / 2;
```

```
    mergeSort (arr, lo, mid);
```

```
    mergeSort (arr, mid+1, hi);
```

```
    mergeTwoSortedArrays (arr, lo, mid, hi);
```

```
}
```

S.C $\rightarrow N + \log N$

$= O(N)$.

```
void mergeTwoSortedArrays (int arr[], int lo,
                           int mid, int hi)
```

$lo \rightarrow mid$; $mid+1 \rightarrow hi$;

```
int ans[hi-lo+1];  $\forall i$  ans[i] = 0;
```

```
i = lo; ; j = mid+1;
```

```
idx = 0;
```

```
while ( i  $\leq$  mid && j  $\leq$  hi ) {
```

```
    if (arr[i]  $\leq$  arr[j]) {
```

```
        ans[idx] = arr[i];
```

```
        i++; idx++;
```

```
    } else {
```

```
        ans[idx] = arr[j];
```

```
        j++; idx++;
```

```
    }
```

```
while ( i  $\leq$  mid ) {
```

```
    ans[idx] = arr[i]; i++;
```

```
    idx++;
```

```
}
```

```
while ( j  $\leq$  hi ) {
```

```
    ans[idx] = arr[j]; j++;
```

```
    idx++;
```

```
}
```

k=0;

for (i=lo; i<=hi; i++) {

|

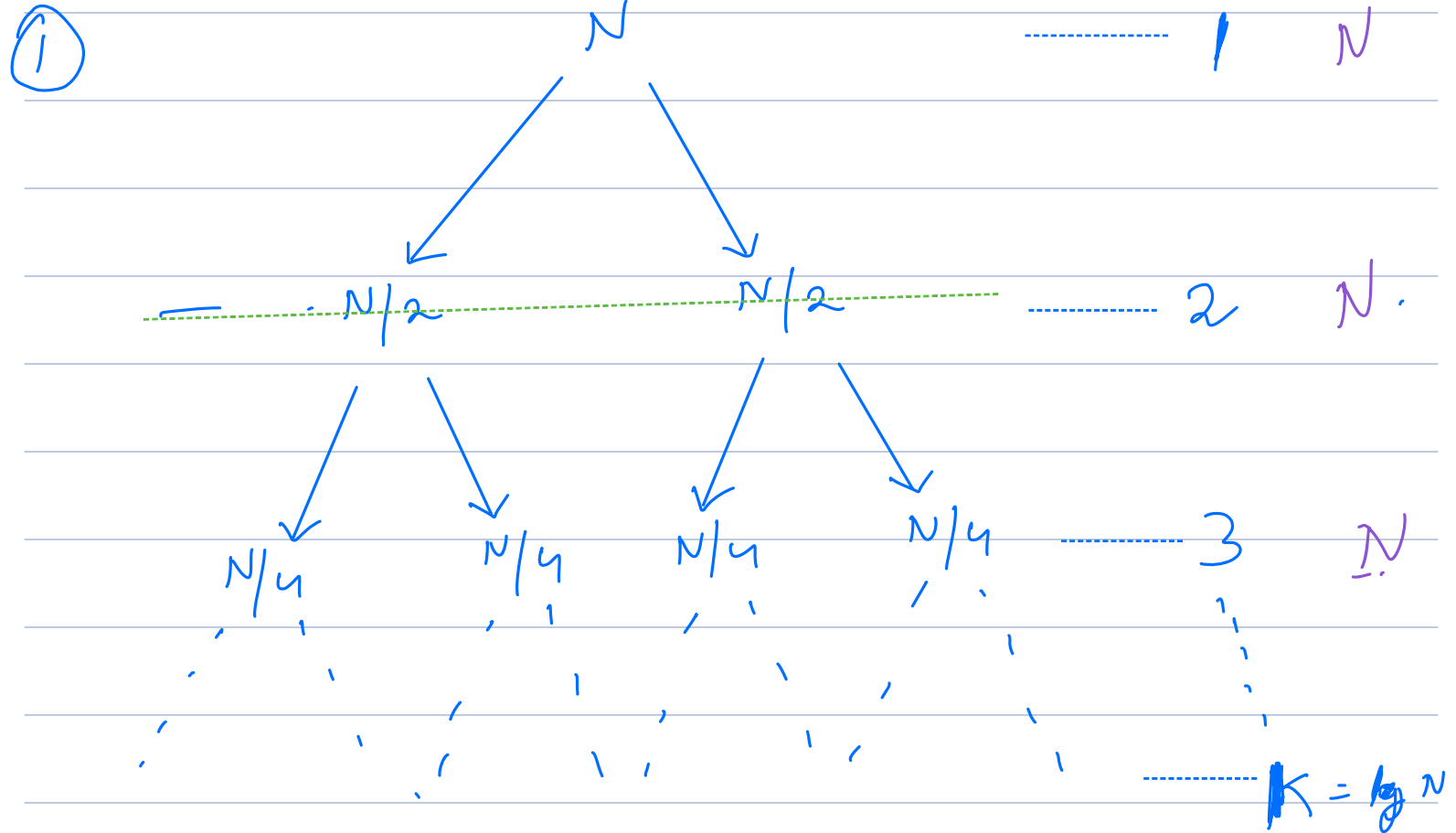
3

ans[i] = ans[k];

k++;

3

Time Complexity Analysis [Merge Sort]



$$N + N + N + \dots + N$$

$\log N$

$$T.C = O(N \cdot \log N)$$

$$2^0 + 2^1 + 2^2 + 2^3 + 2^4 + \dots + 2^{\log k.}$$

N

$\downarrow$
 N

N

N

N

$\dots$

N

$N \cdot \log N$

② Substitution Method

Let us say that
T.C. of Merge Sort of an array
of size N be $\underline{T(N)}$.

$$\begin{aligned} T(N) &= T(N/2) + T(N/2) + N \\ &= 2T(N/2) + N \quad \text{--- (i)} \end{aligned}$$

Replacing N with $N/2$

$$T(N/2) = 2T(N/4) + N/2 \quad \text{--- (2)}$$

Replacing the value of $N/2$
in Statement (i)

$$\begin{aligned} T(N) &= 2(2T(N/4) + N/2) \\ &\quad + N \\ &= 4T(N/4) + N + N \end{aligned}$$

$$T(N) = 4T(N/4) + 2N \quad \text{--- (3)}$$

$$T(N/4) = 2T(N/8) + N/4$$

Substituting the value of $T(N/4)$
in eq 3

$$\begin{aligned} T(N) &= 4 \left(2T(N/8) + N/4 \right) + 2N \\ &= 8T(N/8) + N + 2N \end{aligned}$$

$$T(N) = 8T(N/8) + 3N$$

$$T(N) = 16T(N/16) + 4N$$

$$= 32T(N/32) + 5N$$

$$= 64T(N/64) + 6N$$

$$= 2^k T\left(\frac{N}{2^k}\right) + k \cdot N$$

At some point, $N/2^k = 1$

$$k = \log_2 N$$

$$= 2^{\log_2 N} T(1) + \log_2 N \cdot N$$

$$= N + N \cdot \log_2 N$$

$$T.C = \underline{\underline{O(N \cdot \log N)}}.$$